Ansible Loops:

- Using loops saves administrators from the need to write multiple tasks that use the same module.
- For example, instead of writing five tasks to ensure five users exist, you can write one task that iterates over a list of five users to ensure they all exist.
- Ansible supports iterating a task over a set of items using the loop keyword.
- You can configure loops to repeat a task using each item in a list, the contents of each of the files in a list, a generated sequence of numbers, or using more complicated structures.
- A simple loop iterates a task over a list of items.
- The `loop` keyword is added to the task, and takes as a value the list of items over which the task should be iterated.
- The `loop variable item` holds the value used during each iteration.

EXAMPLES

- This is a simple playbook that is used to check status of services on managed hosts.

```yaml
- name: start Services
  hosts: rhelnode
  tasks:
    - name: httpd is running
      service:
        name: httpd
        state: started
    - name: firewalld is running
      service:
        name: firewalld
        state: started
```

- With `loop` playBook can be written as:

```yaml
- name: Start Services
  hosts: rhelnode
  tasks:
    - name: Checking Services
      service:
        name: "{{ item }}"
        state: started
      loop:
        - httpd
        - firewalld
```

- The list used by loop can be provided by a variable. In the following example, the variable `myservices` contains the list of services that need to be running.

```yaml
- name: Start Services
  hosts: rhelnode
  vars:
    myservices:
      - httpd
      - firewalld
  tasks:
    - name: Checking status and starting
      service:
        name: "{{ item }}"
        state: started
      loop: "{{ myservices }}"
```

Loops over a List of Hashes or Dictionaries:

- The `loop` list does not need to be a list of simple values.
- Each item in the list can be actually a `hash` or a `dictionary`.

EXAMPLE:

```bash
- name: Install and enable services
  hosts: rhelnode
  tasks:
    - name: Add multiple packages with versions
      yum:
        name: "{{ item.name }}"
        state: "{{ item.state }}"
      loop:
        - { name: "httpd", state: "latest" }
        - { name: "vim", state: "present" }
```

- The value of each key in the current item loop variable can be retrieved with the `item.name` and `item.state` variables, respectively.

Earlier-Style Ansible Loops

with_items

- Behaves the same as the loop keyword for simple lists, such as a list of strings or a list of hashes/dictionaries.
- Unlike `loop`, if lists of lists are provided to `with_items`, they are flattened into a single- level list.
- The loop variable item holds the list item used during each iteration.

with_file

- This keyword requires a list of control node file names.
- The loop variable item holds the content of a corresponding file from the file list during each iteration.

with_sequence

- Instead of requiring a list, this keyword requires parameters to generate a list of values based on a numeric sequence.
- The loop variable item holds the value of one of the generated items in the generated sequence during each iteration.

Examples:

- **with_items**
- **with_file**
- **with_sequence**

Each example is self-contained so you can drop it directly into your lab.

1. Playbook Example: with_items

```yaml
- name: Demo of with_items
  hosts: all
  gather_facts: no

  tasks:
    - name: Install multiple packages
      yum:
        name: "{{ item }}"
        state: present
      with_items:
        - httpd
        - vim
        - git

    - name: Loop over list of dictionaries
      debug:
        msg: "User {{ item.name }} has UID {{ item.uid }}"
      with_items:
        - { name: "muhammad", uid: 1001 }
        - { name: "ahmed", uid: 1002 }
```

```

---

**2. Playbook Example: with_file**

**Requirements:**

Create files on the **control node**:

files/msg1.txt

files/msg2.txt

**Playbook:**

```yaml
- name: Demo of with_file
  hosts: all
  gather_facts: no
  tasks:
    - name: Read contents of multiple files
      debug:
        msg: "File content: {{ item }}"
      with_file:
        - files/msg1.txt
        - files/msg2.txt
```

✓ item contains the **file content**, not the filename.

```

3. Playbook Example: with_sequence

```yaml
- name: Demo of with_sequence
  hosts: all
  gather_facts: no
  tasks:
    - name: Create numbered directories
      file:
        path: "/tmp/dir{{ item }}"
        state: directory
      with_sequence: start=1 end=5

    - name: Create even-numbered files
      file:
        path: "/tmp/file{{ item }}"
        state: touch
      with_sequence: start=0 end=10 stride=2
    - name: Alphabet sequence
      debug:
        msg: "Letter: {{ item }}"
      with_sequence: start=A end=E
```
